Cmpe 492 - Presentation

Mustafa Ocak - Halil İbrahim Kasapoğlu
Advisor: Atay Ozgovde

Bogazici University

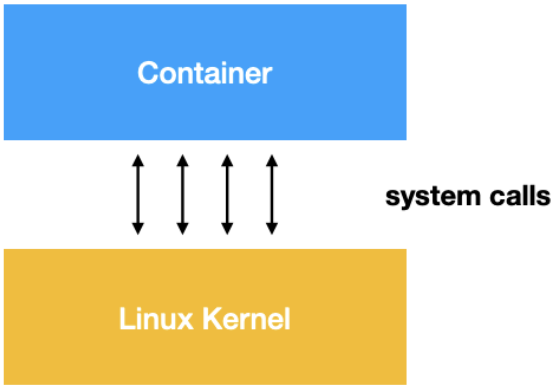
Security Concerns

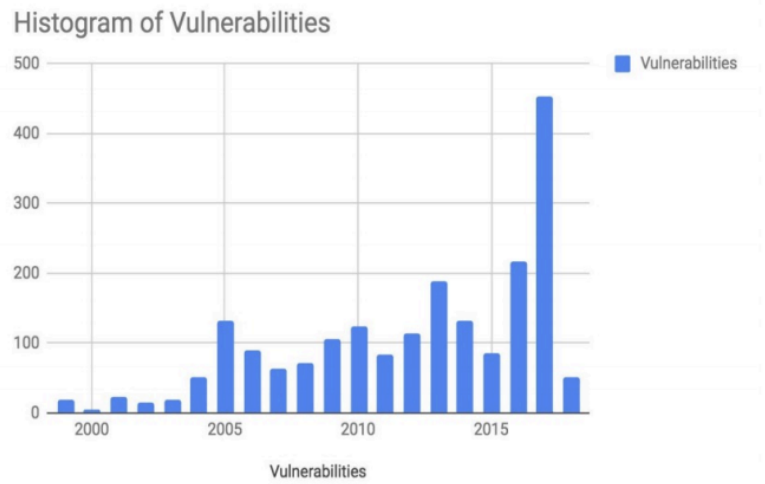
Each application running in a virtual machine runs on its own guest operating system, OS-level virtualization allows multiple tenants to efficiently share a common OS. Although efficient, such sharing is also a security concern; as Daniel Walsh quipped, “containers do not contain”. If the OS itself does not run in the container, OS resources with incomplete virtualization are vulnerable and kernel bugs can be exploited through a large attack surface (over 300 system calls).

Container/Kernel Attack

Apps/Containers Attack Kernel

Through system calls

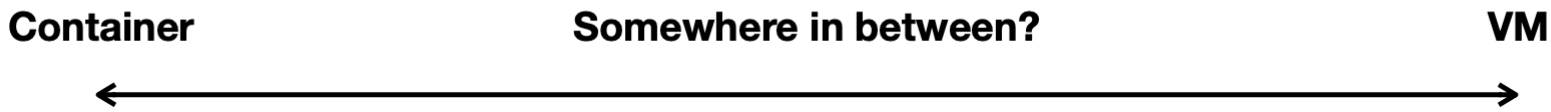




2046 Common Vulnerabilities and Exposures (CVE) since 1999

gVisor Concept

Combining the lightweight efficiency of containers with the robust security of virtual machines: An Application Kernel to Intercept Common Syscalls



1. What Does It Mean to Run as Root?

- The **root user** has the highest level of privileges on a Unix-based system.
- Root can:
  - Read, write, and delete any file.
  - Start or stop system services.
  - Change user permissions and manage hardware.
- When an application runs as root, it inherits these unrestricted privileges.
- This can be dangerous if the application is exploited or contains bugs.

Malicious Software or Bugs

- What if there is a bug or malicious software that runs:

Code Example

```
$ rm -rf $(UNDEFINED_DIR)/*
```

- If `UNDEFINED_DIR` is empty or not set, this becomes:

Expanded Code

```
$ rm -rf /*
```

- This could delete all files and directories on the system, including critical ones like `/bin`, `/etc`, `/home`, etc.

Limit Privileges

- Permission of kernel calls are checked before execution
- Limited File System Access
- Limited devices access including network devices

2. Other Roles or Users

- **Normal Users**:
  - Limited access to files and processes.
  - Can only modify files they own or have permission for.
  - Used for daily activities to avoid accidental system damage.
- **System Users**:
  - Special accounts created for system services (e.g., ‘www-data‘ for web servers).
  - Have restricted access to only what their service requires.
- **Groups**:
  - A way to share privileges among multiple users.
  - Example: Developers in the ‘dev‘ group share access to source code.

What Are Linux Capabilities?

- Capabilities divide the all-powerful `root` privilege into smaller, specific privileges.
- Allow processes to have only the permissions they need, enhancing security.
- Example: A process can be granted permission to bind to privileged ports (`CAP_NET_BIND_SERVICE`) without full `root` access.

Common Linux Capabilities

Capability	Description
`CAP_SYS_ADMIN`	Perform administrative tasks like mounting filesystems or changing namespaces.
`CAP_NET_ADMIN`	Modify network settings, including routing tables and interfaces.
`CAP_NET_BIND_SERVICE`	Bind to ports below 1024 (privileged ports).
`CAP_CHOWN`	Change ownership of files (normally restricted to `root`).
`CAP_SETUID`	Change the user ID of the process (e.g., switch to another user).
`CAP_SETGID`	Change the group ID of the process.
`CAP_SYS_TIME`	Set the system clock or real-time clock.

Understanding `CAP_SYS_ADMIN`

- Considered the "supercapability" because it covers many administrative tasks.
- Examples of tasks:
  - Mounting and unmounting filesystems.
  - Changing namespaces.
  - Accessing kernel debugging facilities.
- **Risks**: Nearly equivalent to giving full `root` access.

How to Use Capabilities

- **View Capabilities**:
  - `getcap /path/to/file`
- **Assign Capabilities**:
  - `sudo setcap cap_net_bind_service=+ep /usr/bin/python3`
- **Remove Capabilities**:
  - `sudo setcap -r /path/to/binary`

Advantages and Drawbacks

Advantages:

- Granular control over privileges.
- Limits the impact of a compromised process.
- Follows the principle of least privilege.

Drawbacks:

- Misconfiguration can create vulnerabilities.
- Not all privileged operations are covered by capabilities.

What are Cgroups?

- **Cgroups (Control Groups)** are a Linux kernel feature that allows fine-grained control over system resources.
- They are used to:
  - Allocate, limit, and monitor CPU, memory, disk I/O, and network bandwidth.
  - Isolate processes into groups with specific resource constraints.

Key Features of Cgroups

- **Resource Limiting**: Restrict how much CPU, memory, or disk I/O a process can use.
- **Prioritization**: Assign more resources to higher-priority processes.
- **Accounting**: Monitor resource usage in real-time.
- **Isolation**: Ensure processes do not interfere with each other.
- **Dynamic Control**: Add or remove processes from cgroups without restarting.

How Cgroups Work

- **Hierarchy**: Organized into a tree structure; child cgroups inherit limits from their parent.
- **Subsystems (Controllers)**:
  - `cpu`: Controls CPU usage.
  - `memory`: Manages memory limits.
  - `blkio`: Controls disk I/O bandwidth.
  - `net_cls` and `net_prio`: Manage network traffic.
- Configured via `/sys/fs/cgroup` or tools like `systemd`.

Example: Limiting Memory Usage

Steps to limit a process to 512 MB of memory:

1. Create a cgroup: `sudo cgcreate -g memory:/mygroup`.
2. Set a memory limit: `sudo cgset -r memory.limit_in_bytes=512M mygroup`.
3. Run a process in the cgroup: `sudo cgexec -g memory:/mygroup my_program`.

Advantages and Disadvantages

Advantages:

- Efficient resource management in multi-tenant systems.
- Prevents resource starvation from misbehaving processes.
- Integral to containerization technologies (e.g., Docker, Kubernetes).

Disadvantages:

- Misconfiguration can create resource bottlenecks.
- Complexity in managing and setting up cgroups.
- Older cgroups (v1) lacked consistency; resolved in cgroups v2.

What are Namespaces?

- **Namespaces** are a Linux kernel feature that isolate and virtualize system resources for a group of processes.
- They provide:
  - **Isolation**: Processes in different namespaces are isolated from each other.
  - **Virtualization**: Each namespace creates a unique view of a system resource.
- Key feature of modern containerization technologies like Docker and Kubernetes.

Types of Namespaces (1/2)

Namespace Type	Description
`pid`	Isolates the process ID (PID) space. Processes in one namespace cannot see or interact with processes in another.
`net`	Isolates network interfaces, routing tables, and sockets. Each namespace can have its own IP address and routes.
`mnt`	Isolates filesystem mount points. Each namespace can mount or unmount filesystems without affecting others.
`uts`	Isolates hostname and domain name settings. Each namespace can have its own hostname.

Types of Namespaces (2/2)

Namespace Type	Description
`ipc`	Isolates inter-process communication (e.g., shared memory).
`user`	Isolates user and group IDs. Processes in one namespace can have a different UID/GID mapping than others.
`cgroup`	Isolates the cgroup hierarchy, enabling separate resource limits for each namespace.

How Namespaces Work

- Each namespace provides a separate "view" of a system resource.
- Processes in the same namespace share the same view, while processes in different namespaces see isolated environments.
- Example:
  - A container with its own `net` namespace has its own IP address and routes, separate from the host.
- Configured using system calls like `unshare`, `setns`, or tools like `docker` and `ip`.

Example: Isolating a Network Namespace

Steps to create and use a network namespace:

1. Create a namespace: `sudo ip netns add mynamespace`.
2. Assign an interface: `sudo ip link set dev eth0 netns mynamespace`.
3. Enter the namespace: `sudo ip netns exec mynamespace bash`.
4. Configure the namespace:
   - Assign an IP: `sudo ip addr add 192.168.1.100/24 dev eth0`.
   - Bring up the interface: `sudo ip link set dev eth0 up`.

Advantages and Disadvantages

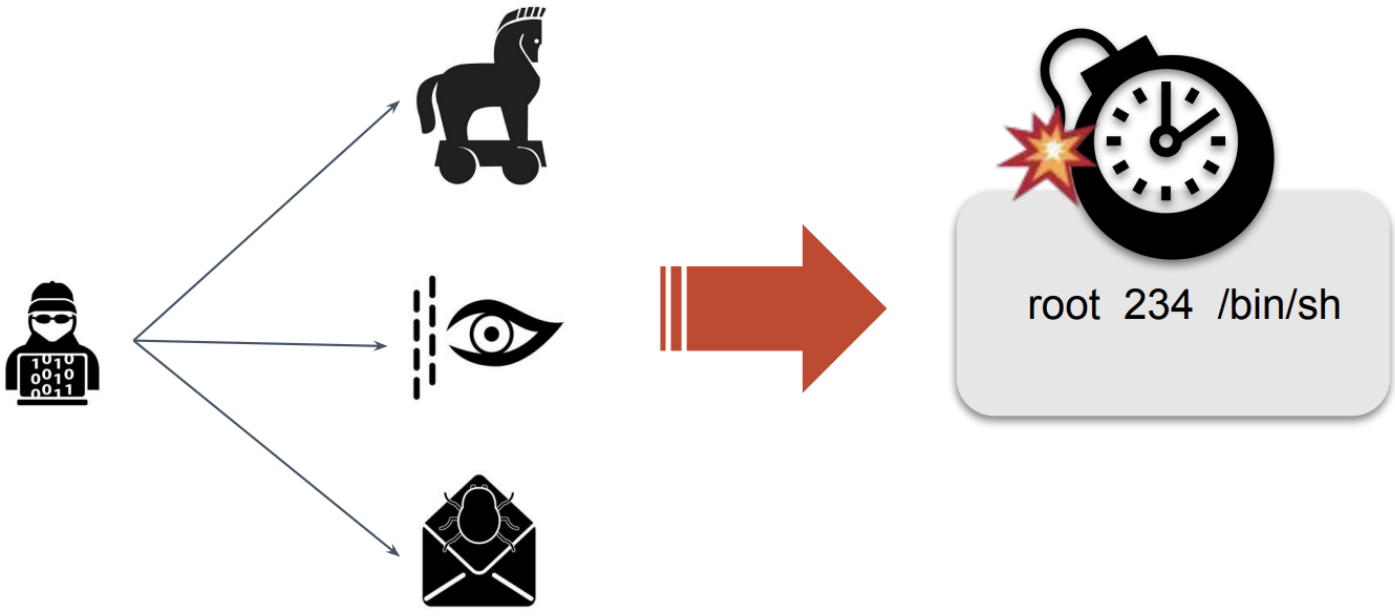
Advantages:

- Provides strong isolation for processes and resources.
- Enables lightweight virtualization without the overhead of full VMs.
- Essential for containerization technologies like Docker and Kubernetes.

Disadvantages:

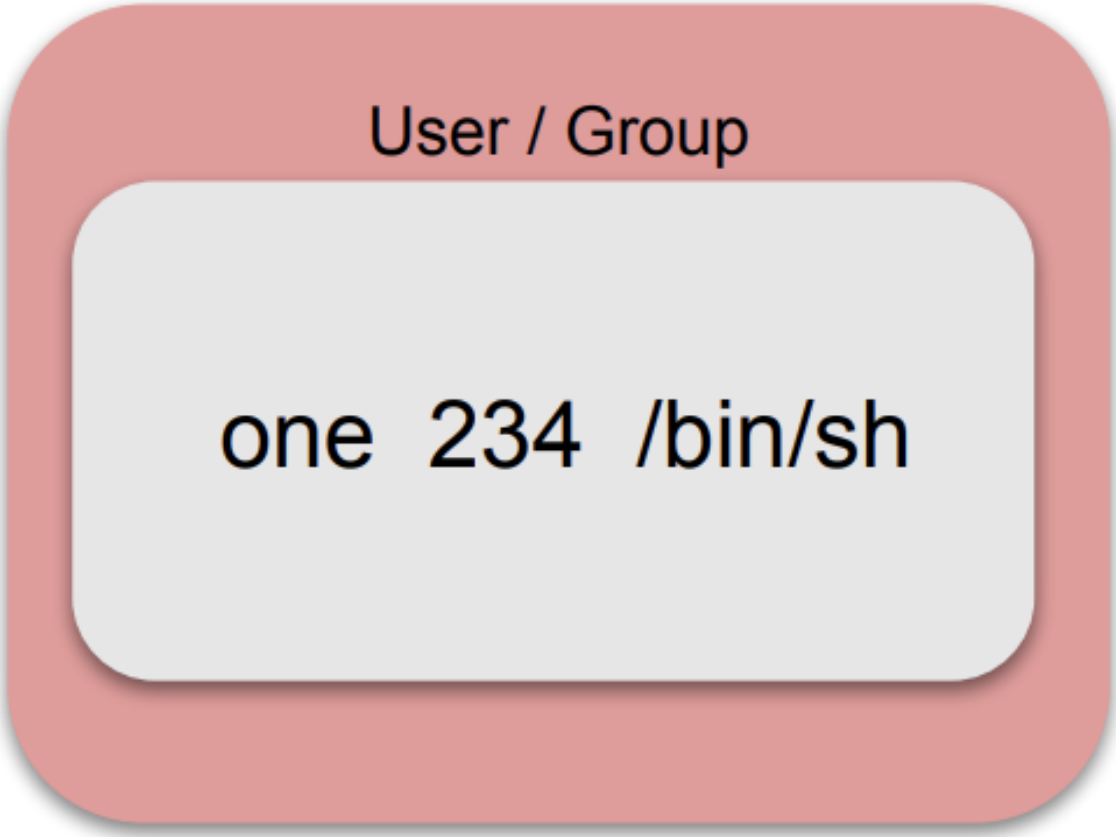
- Complex to set up manually.
- Processes in the host namespace can still access resources in other namespaces unless explicitly blocked.

Container as Root



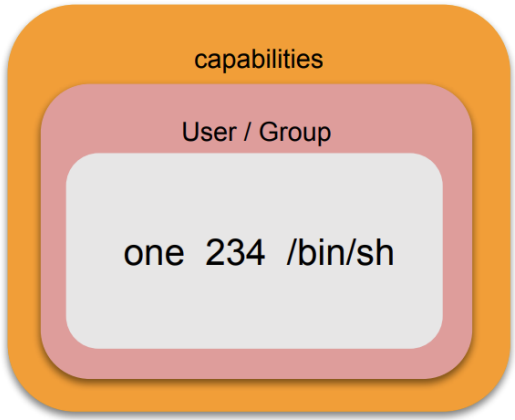
Nothing shields the system!

CAP_SETUID ?



Drop Capabilities

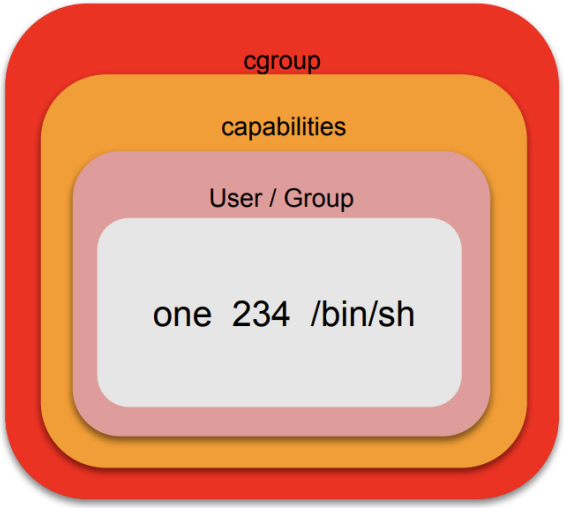
- Examples of dropped capabilities:
  SYS_MODULE, SYS_ADMIN, SYS_TIME,
  SYS_RESOURCE, NET_ADMIN, SYS_LOG, …
- Fewer capabilities, better isolation!



Restrict Hardware Access - Cgroups

- Cgroup limits, accounts for, and isolates the resource usage:
  - cpu - limits access to the CPU
  - cpuacct - accounts cpu usage by cgroup
  - cpuset - assign cores & memory nodes to cgroup
  - devices - control device access by cgroup
  - memory - limits & accounts memory usage

  and more

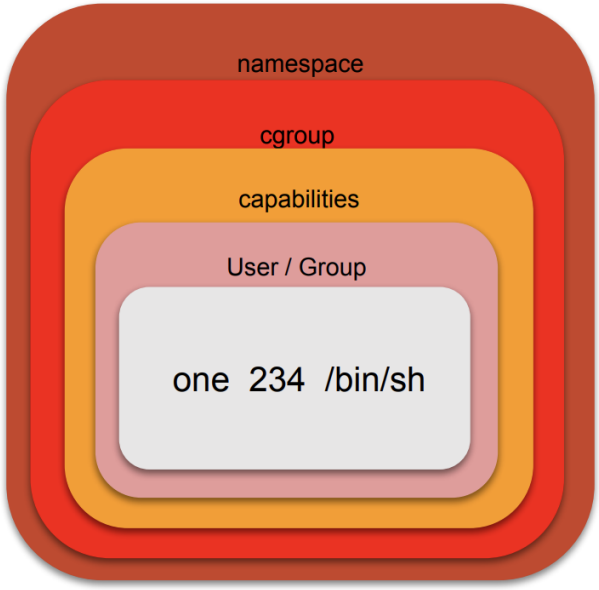


But still can see all processes, network interfaces, mount points on the system!

Restrict what can process see- Apply Namespaces

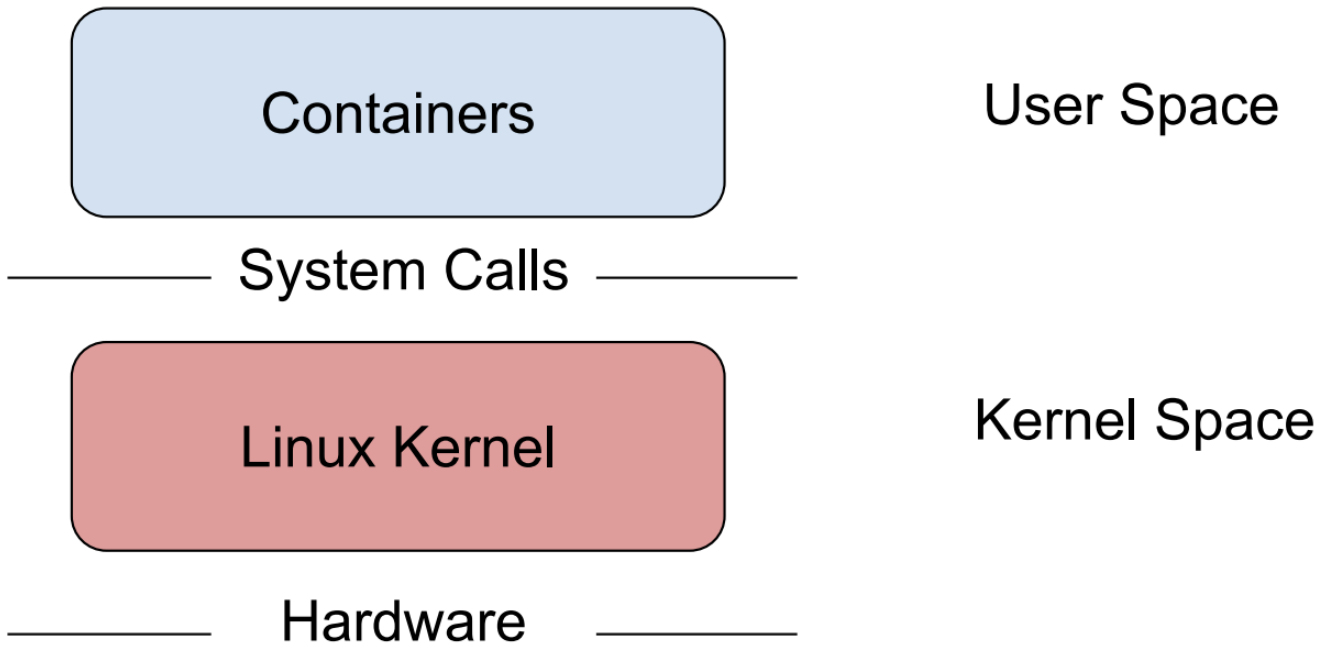
- Provide isolation for each namespace type
- Currently support 7 different namespaces:

  Network, PID, mount, user, IPC, UTS, cgroup
- More to come



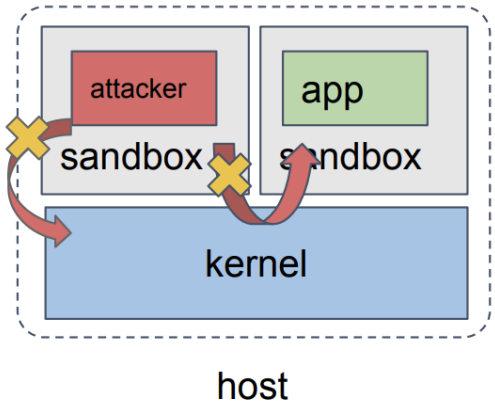
Container Isolation

At the core, container syscalls to kernel, if we don't add a layer between them.



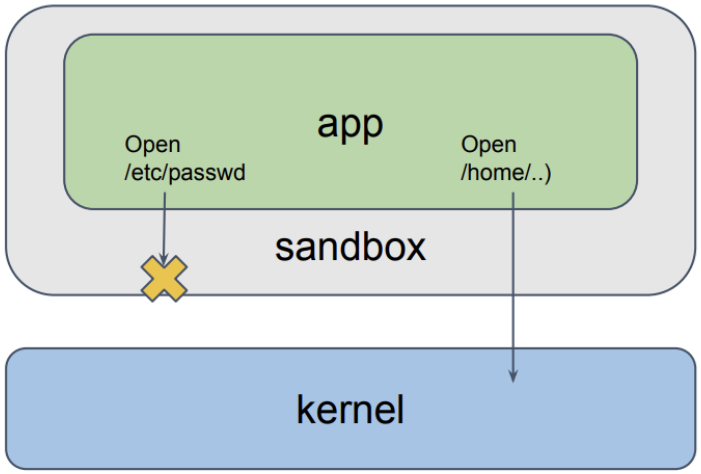
Sandbox

- Sandbox is an effective layer to reduce the attack surface.



Rule Based Sandbox

- AppArmor, SELinux, Seccomp-bpf

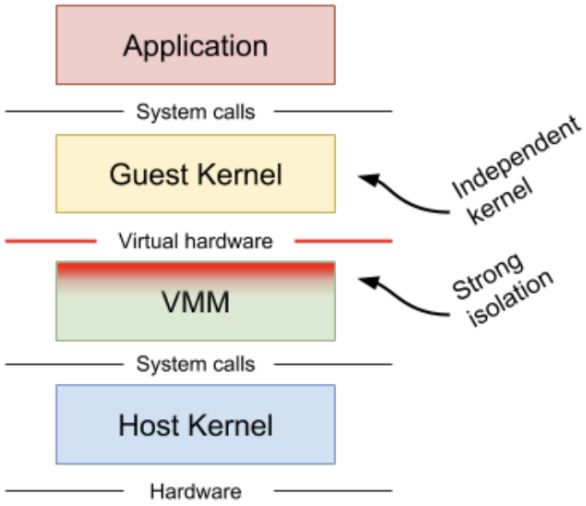


- Complexity in Rule Configuration
- Dynamic Application Behavior
- Interdependencies
- Unanticipated Threats
- Maintenance Overhead

Conventional VM

Recap: Conventional VM

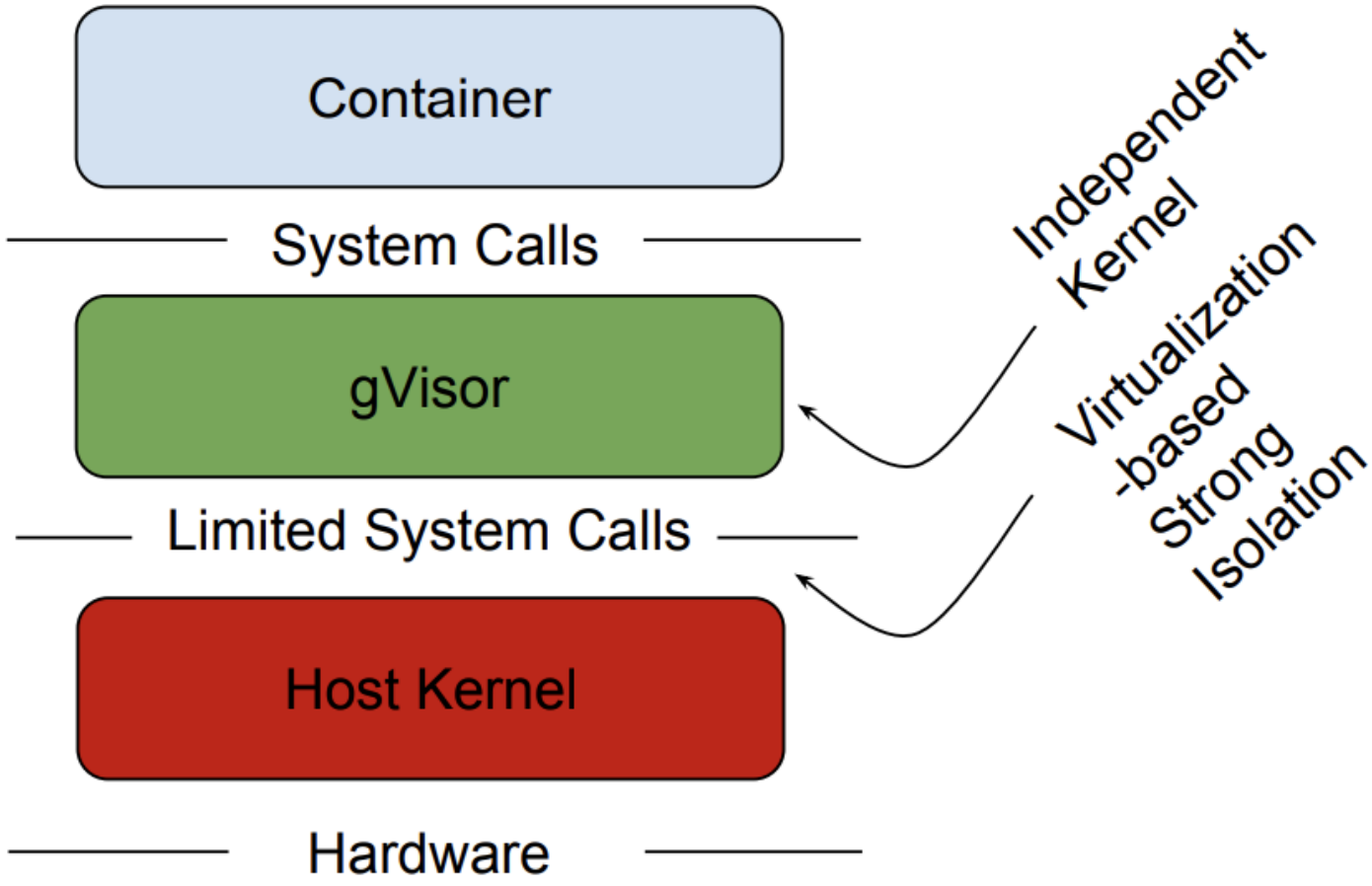
On the other end



- Independent guest kernels
- Virtual hardware interface
  - Clear privilege separation and state encapsulation
- **But virtualized hardware interface is inflexible**
  - e.g., can't change number of virtualized cores at run time
- and **VM is heavy weight**

Q: Why/how this extra guest OS layer really helps improve security?

gVisor


Container
System Calls
gVisor
Limited System Calls
Host Kernel
Hardware
Independent Kernel
Virtualization -based Strong Isolation

gVisor

gVisor is a container runtime that enhances security by acting as a user-space kernel. It intercepts system calls from containers and implements its own kernel functionality, isolating containers from the host system.

Key Features:

- **System Call Interception:** Acts as a middle layer between containers and the host kernel.
- **Virtualization-Based Strong Isolation:** Provides strong isolation without the overhead of full VMs.
- **Reduced Attack Surface:** Only limited system calls are passed to the host kernel.
- **Lightweight:** Compatible with Docker and Kubernetes, offering efficient security.

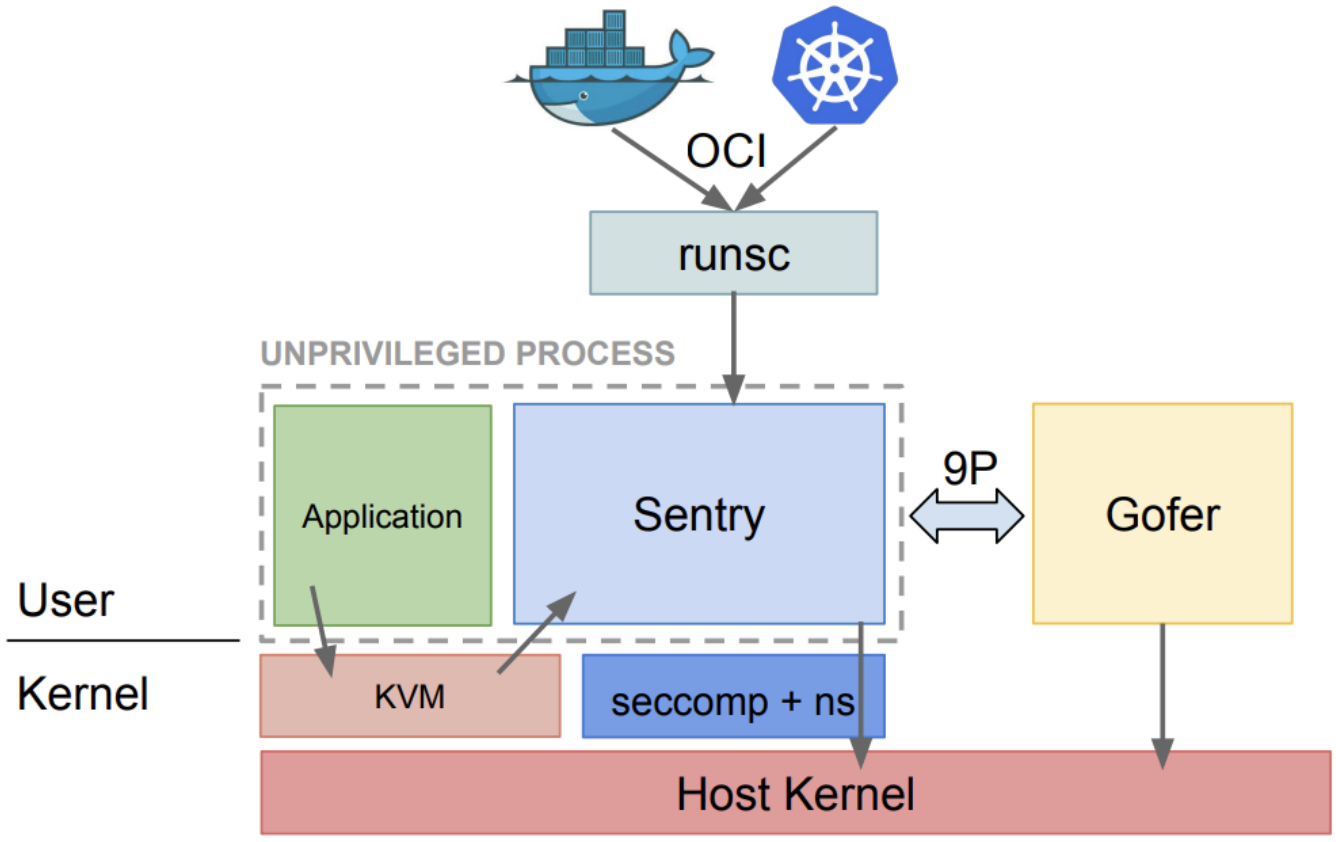
gVisor Architecture

gVisor provides strong application isolation by acting as a user-space kernel. It intercepts system calls, enforces security policies, and decouples applications from the host kernel.

Key Components:

- **OCI and ‘runsc‘:** ‘runsc‘ acts as a runtime interface, compatible with Docker and Kubernetes.
- **Sentry:** The user-space kernel that intercepts and validates system calls.
- **Gofer:** Manages file system operations using the 9P protocol.
- **Seccomp + Namespaces:** Lightweight isolation mechanisms for syscall filtering and resource isolation.
- **KVM:** Sentry to act as both guest OS and VMM
- **Host Kernel:** The underlying Linux kernel is protected by gVisor’s layers.

gVisor Architecture



gVisor Compatability

gVisor implements the OCI (Open Container Initiative) standard, used by Docker. Thus, Docker users can readily switch between the default engine, runc (based on Linux namespaces and cgroups), and the gVisor runtime engine, named runsc.

gVisor Architecture

The application runs on the Sentry, which both implements Linux and itself runs on Linux. This Linuxon-Linux architecture provides defense in depth. A malicious application that exploits a bug in the Sentry only gains access to a restricted user-space process; in contrast, a process escaping a Docker runc container might gain root access.

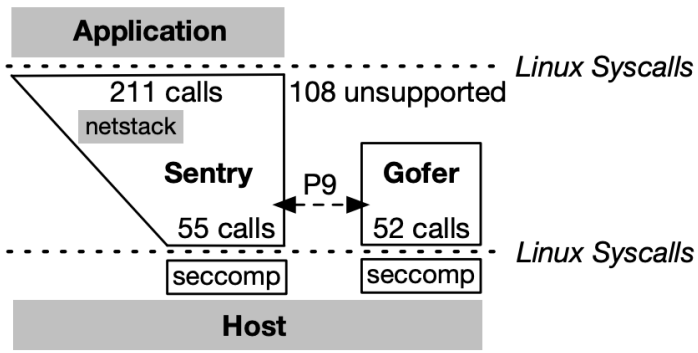


Figure 1: **gVisor Architecture.**

System Calls in Sentry

The KVM backend uses hardware-assisted virtualization to run the container workload inside a lightweight virtual machine (VM). This leverages KVM (Kernel-based Virtual Machine) for stronger isolation. The Systrap backend runs in user space, relying on the host Linux kernel without needing hardware virtualization. It uses ptrace and seccomp-bpf to intercept and emulate system calls. seccomp filters prevent a compromised Sentry from using system calls beyond the 55 it is expected to use. The gVisor engineers identified open and socket as common attack vectors so these calls are not among the whitelisted 55.

Storage - 1

There are three main patterns for how the Sentry handles file-related system calls. First, gVisor implements several file systems internally (e.g., a tmpfs, procfs, and overlayfs); the Sentry can generally serve I/O to these internal file systems without calling out to the host or other helpers.

Storage - 2

Second, the Sentry services open calls to external files (as permitted) with the help of a separate process called the Gofer; the Gofer is able to open files on the Sentry's behalf and pass them back via a 9P (Plan 9) channel.

Storage - 3

Third, after the Sentry has a handle to an external file, it can serve read and write system calls from the application by issuing similar system calls to the host. If gVisor is running in KVM mode, the Sentry must switch from GR0 (Guest Ring 0) to HR3 (Host Ring 3) to process such requests, as system calls to the host cannot be invoked directly from GR0

Networking

The network stack (called netstack) is implemented in the Sentry in Go; netstack directly communicates via a raw socket with a virtual network device.

Memory

A traditional OS running on a virtual machine assumes a view of "physical" memory, from which it allocates memory for processes. In contrast, the Sentry is built to run on Linux and implements process-level mmap calls by itself invoking mmap on the host.

Questions

- **Container Lifecycle:** How quickly gVisor can start and stop containers?
- **System Call Overheads:** The overheads associated with servicing system calls?
- **Networking Performance:** Evaluation of gVisor's user-space, Go-based network stack.
- **File Access Overheads:** Analysis of file access performance in gVisor.
- **Python Module Import Performance:** Effects of gVisor's system call and I/O overheads on Python module imports .

Container Lifecycle

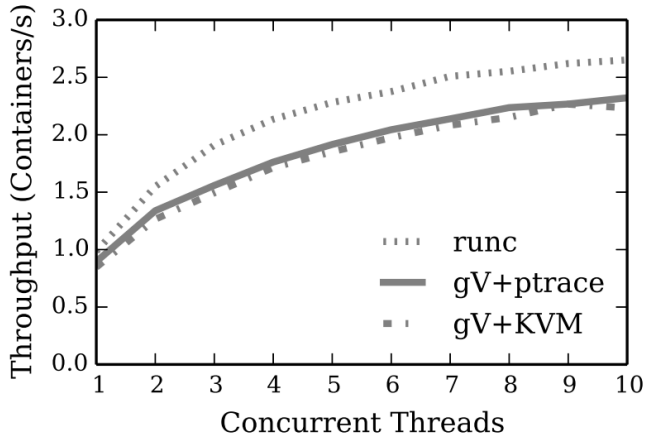


Figure 2: **Container Init/Teardown Performance.** *Varying numbers of threads (x-axis) create and destroy containers concurrently. The total system throughput is reported as containers-per-second on the y-axis.*

- With no concurrency, the total setup and teardown times are 1.014s for runc, 1.117s for gVisor+KVM, and 1.181s for gVisor+ptrace.
- runc achieves at most 2.6 containers/second, and runsc achieves 2.3 containers per second

System Calls

Depending on the system call, the Sentry may handle it internally, invoke a call on the host, or get help from the Gofer. (gettimeofday) syscall tested on every combination of runtime mode (KVM or ptrace) and implementation pattern (Sentry only, invoke on host, and get help from Gofer). runc is only 32the fastest gVisor result (Sentry-only on KVM) is 2.8× slower. In KVM mode, calling to the host is 9× slower than operating just within the Sentry, and calling to the Gofer is 72× slower. ptrace latencies are 42-232× slower than native latencies.

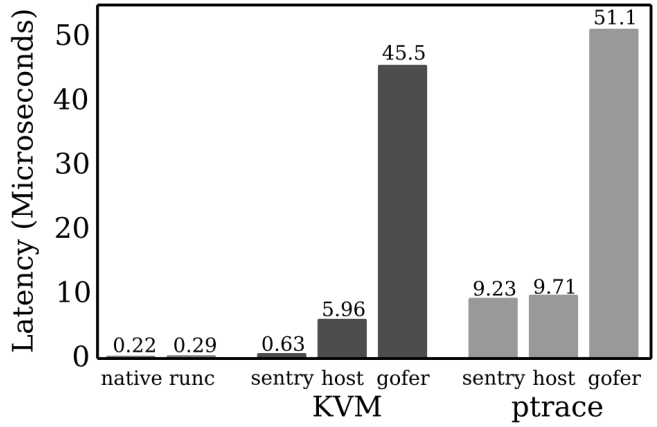


Figure 3: **System Call Overhead.** *The bars show the average latency for* `gettimeofday` *across 100M executions.*

Network

The socket system call was identified as a commonly exploited attack vector by the gVisor team; gVisor guards against such attacks by relying on a user-space networking stack, called netstack. We evaluate the network performance by downloading files of varying sizes (from 1 MB to 1 GB) from a speedtest site using wget.

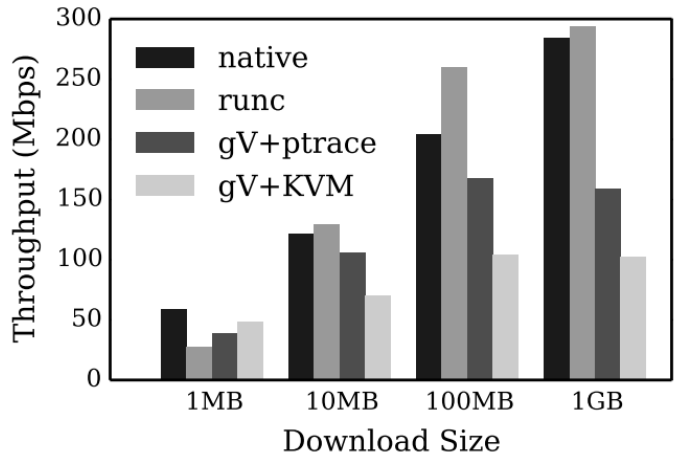


Figure 5: **Network Throughput.** *Throughput is measured by downloading files of varying sizes (x-axis) with* `wget`.

Storage

The latency of opening and closing a file without performing any I/O requests was observed.

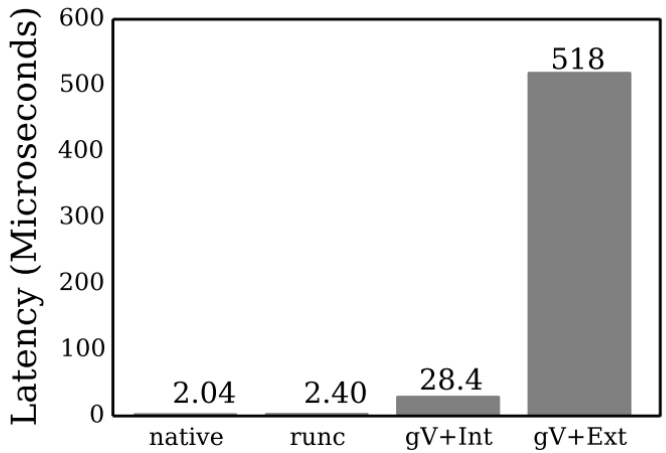


Figure 6: **Open and Close Latency.** *Results are an average over 100K consecutive accesses to the same file on native, runc, internal tmpfs (gV+Int), and external tmpfs (gV+Ext).*

the write size. ws, opening and closing a file on an external tmpfs is 216× slower than making the same accesses from a runc container, suggesting routing through the Gofer is a major bottleneck. Accesing a file in the Sentry's internal tmpfs is faster than the external accesses, but is still 12× slower than runc.

Storage Cont'd

Read throughput is observed by performing 100K sequential reads (of varying size) to a large in-memory file. gVisor performs poorly for small requests, but access to the internal tmpfs becomes competitive for request sizes around 64 KB and throughput to an external tmpfs becomes competitive around 1 MB.

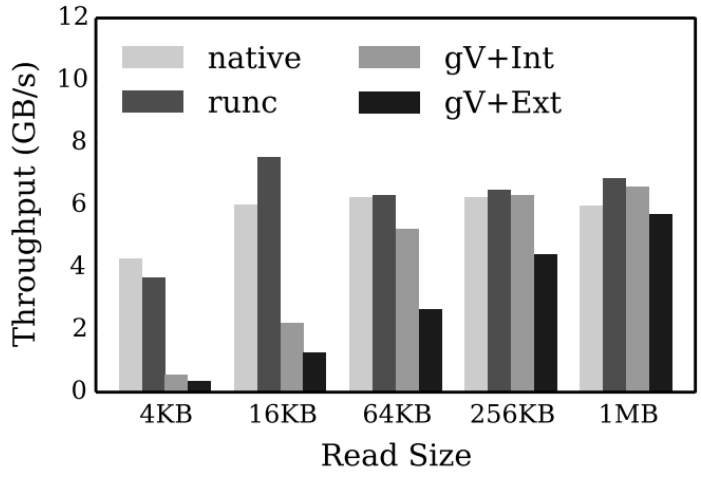


Figure 7: **Read Throughput (tmpfs).** *The x-axis shows the read size. The right two bars in each group are for gVisor.*

Storage Cont'd

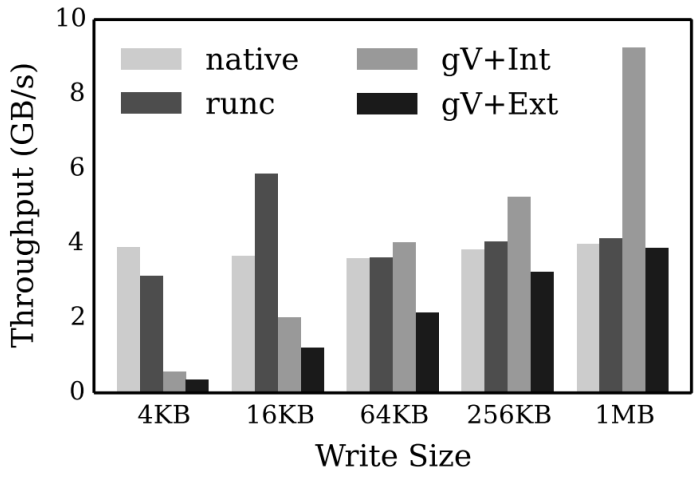


Figure 9: **Write Throughput (tmpfs).** *The x-axis shows the write size.*

Python

It is shown that importing Python modules from the Sentry's prepopulated internal tmpfs is 40% to 70% faster than fetching modules from an external tmpfs. This suggests that Python workloads will benefit from mechanisms that make application resources available inside a container through some means other than the Gofer and the regular I/O stack.

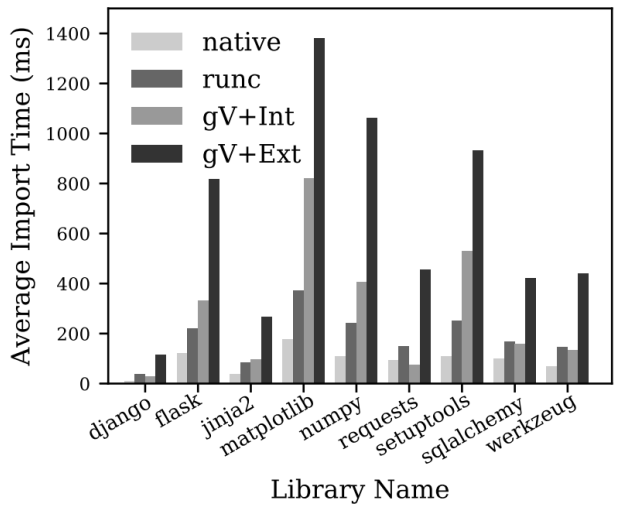


Figure 10: **Python Import Latency.** *We measure the time to import various modules (along x-axis) without containers, with runc, and with gVisor. With containers, we show performance with a host tmpfs volume mount (both runc and gVisor) and Sentry tmpfs mount.*

Python Cont'd

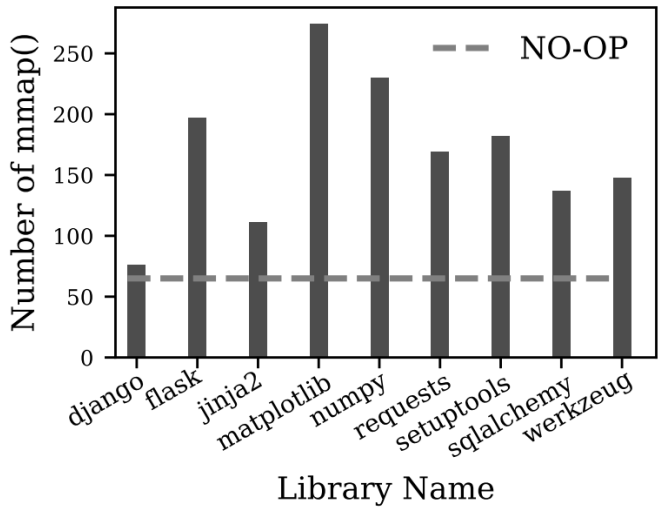


Figure 11: **Number of mmap.** *The dashed line indicates mmap calls during Python startup with no imports (the portion of bars above the line indicates module-specific mmaps).*

Overall results

Truue costs of effectively containing are high: system calls are 2.2 x slower, memory allocations are 2.5 x slower, large downloads are 2.8 x slower, and file opens are 216x slower [1].

Vulnerability Simulation

kubernetes-goat

- **Purpose**: A deliberately vulnerable Kubernetes cluster for learning and testing security attacks.
- **Covers**: OWASP Top 10 and Kubernetes-specific vulnerabilities.

vulnerable-kubernetes-cluster

- **Purpose**: Deploys a misconfigured Kubernetes cluster for security testing.

Scanning and Auditing Tools

trivy

- **Purpose**: Vulnerability and misconfiguration scanner for Kubernetes and container images.
- **Covers**: OWASP issues like sensitive data exposure, injection risks.

kube-hunter

- **Purpose**: Hunts for security vulnerabilities in Kubernetes clusters.
- **Detects**: Misconfigured API servers, insecure etcd, open kubelet ports.

kube-bench

- **Purpose**: Checks your Kubernetes cluster for CIS Benchmark compliance.

kubescape

- **Purpose**: Scans Kubernetes clusters for misconfigurations and OWASP-related risks.

Exploitation Frameworks

kubesploit

- **Purpose**: Penetration testing tool for Kubernetes clusters.
- **Simulates**: Privilege escalation, lateral movement.

Peirates

- **Purpose**: Kubernetes penetration testing tool.
- **Features**: Tests for RBAC misconfigurations, pod escape vulnerabilities.

CIS Kubernetes Benchmark

The CIS Kubernetes Benchmark is a set of security best practices and configuration guidelines for Kubernetes clusters developed by the Center for Internet Security (CIS). It is designed to help organizations secure their Kubernetes environments by addressing common vulnerabilities and misconfigurations that could be exploited by attackers.

Available for K8S 1.24 for now.

Trivy can generate compliance reports for CIS benchmark

CIS Kubernetes Benchmark

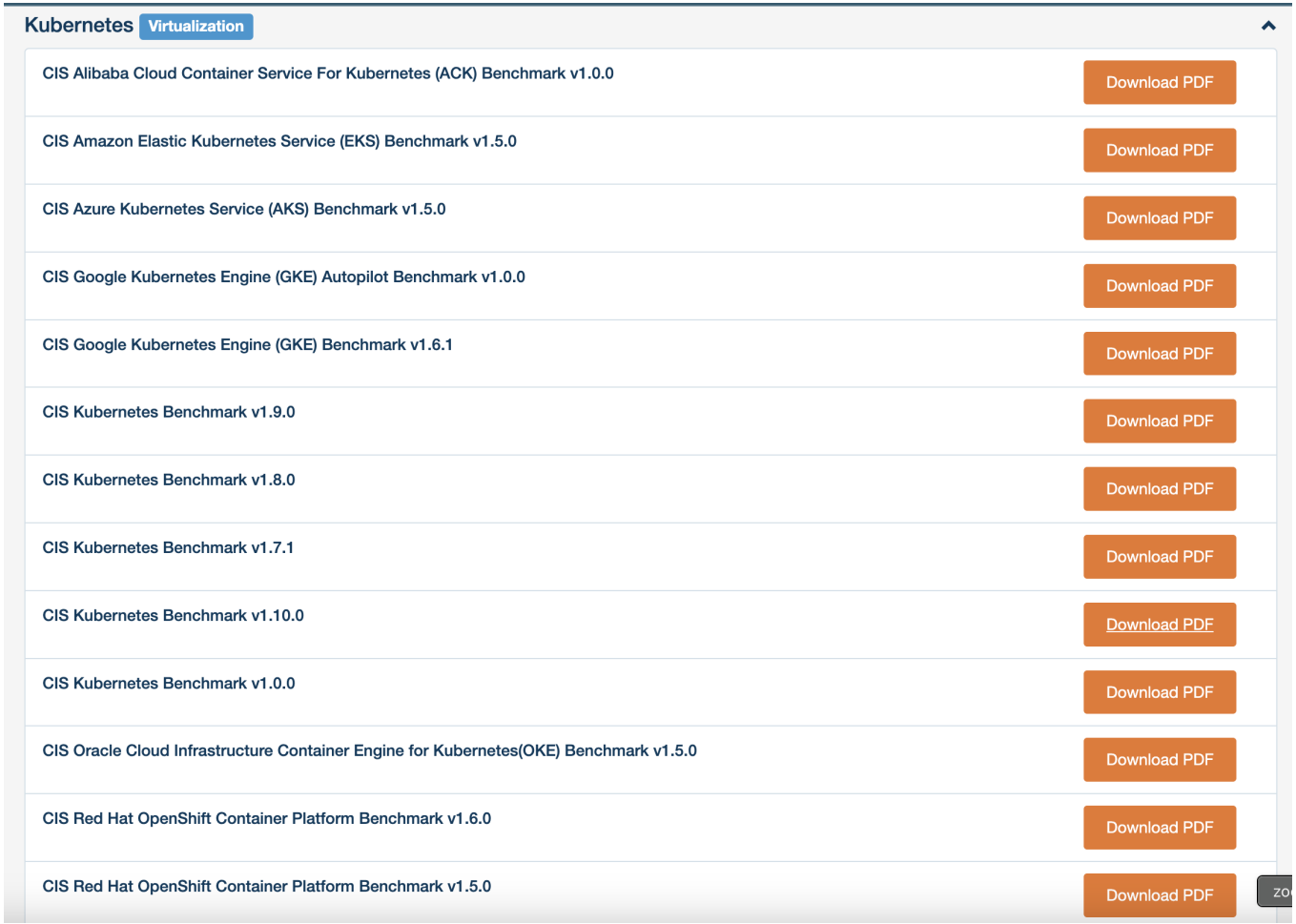


Figure 1: CIS Benchmarks

Kube-Bench CIS Eval

```
[INFO] 1 Master Node Security Configuration
[INFO] 1.1 API Server
[FAIL] 1.1.1 Ensure that the --allow-privileged argument is set to false (Scored)
[FAIL] 1.1.2 Ensure that the --anonymous-auth argument is set to false (Scored)
[PASS] 1.1.3 Ensure that the --basic-auth-file argument is not set (Scored)
[PASS] 1.1.4 Ensure that the --insecure-allow-any-token argument is not set (Scored)
[FAIL] 1.1.5 Ensure that the --kubelet-https argument is set to true (Scored)
[PASS] 1.1.6 Ensure that the --insecure-bind-address argument is not set (Scored)
[PASS] 1.1.7 Ensure that the --insecure-port argument is set to 0 (Scored)
[PASS] 1.1.8 Ensure that the --secure-port argument is not set to 0 (Scored)
[FAIL] 1.1.9 Ensure that the --profiling argument is set to false (Scored)
[FAIL] 1.1.10 Ensure that the --repair-malformed-updates argument is set to false (Scored)
[PASS] 1.1.11 Ensure that the admission control policy is not set to AlwaysAdmit (Scored)
[FAIL] 1.1.12 Ensure that the admission control policy is set to AlwaysPullImages (Scored)
[FAIL] 1.1.13 Ensure that the admission control policy is set to DenyEscalatingExec (Scored)
[FAIL] 1.1.14 Ensure that the admission control policy is set to SecurityContextDeny (Scored)
[PASS] 1.1.15 Ensure that the admission control policy is set to NamespaceLifecycle (Scored)
[FAIL] 1.1.16 Ensure that the --audit-log-path argument is set as appropriate (Scored)
[FAIL] 1.1.17 Ensure that the --audit-log-maxage argument is set to 30 or as appropriate (Scored)
[FAIL] 1.1.18 Ensure that the --audit-log-maxbackup argument is set to 10 or as appropriate (Scored)
[FAIL] 1.1.19 Ensure that the --audit-log-maxsize argument is set to 100 or as appropriate (Scored)
[PASS] 1.1.20 Ensure that the --authorization-mode argument is not set to AlwaysAllow (Scored)
[PASS] 1.1.21 Ensure that the --token-auth-file parameter is not set (Scored)
[FAIL] 1.1.22 Ensure that the --kubelet-certificate-authority argument is set as appropriate (Scored)
```

Figure 2: Kube Bench Running CIS

NSA MITRE

NSA Kubernetes Security Guidance

- The NSA (National Security Agency) has released specific security guidance on how to securely deploy Kubernetes clusters. This guidance addresses critical security aspects, including controlling access to the Kubernetes API, hardening the etcd database, securing network policies, and proper RBAC (Role-Based Access Control) configurations.

MITRE ATTCK for Containers

- The MITRE ATTCK framework has been extended to cover container and Kubernetes security. This includes specific tactics and techniques for containers, such as container escape, privilege escalation, and attacks against the Kubernetes API.
- MITRE helps organizations map their security defenses against known attack patterns in containerized environments.

Kubescape

- **Shift-left security:** Kubescape enables developers to scan for misconfigurations as early as the manifest file submission stage, promoting a proactive approach to security.
- **Runtime security:** Kubescape extends its protection to the runtime environment, providing continuous monitoring and threat detection for deployed applications and runtime context to posture management.
- **IDE and CI/CD integration:** The tool integrates seamlessly with popular IDEs like VSCode and Lens, as well as CI/CD platforms such as GitHub and GitLab, allowing for security checks throughout the development process.
- **Cluster scanning:** Kubescape can scan active Kubernetes clusters for vulnerabilities, misconfigurations, and security issues.

Kubescape

- **Multi-framework compliance support:** Kubescape can test against many security frameworks, including NSA-CISA Kubernetes Hardening Guidance, MITRE ATTCK, SOC 2, CIS Benchmarks, and more. Thus, simplifying compliance efforts and protecting from configuration drift.
- **YAML and Helm chart validation:** The tool checks YAML files and Helm charts for correct configuration according to the frameworks above, without requiring an active cluster.
- **Kubernetes hardening:** Kubescape ensures proactive identification and rapid remediation of misconfigurations and vulnerabilities through manual, recurring, or event-triggered scans.
- **Multi-cloud support:** Kubescape offers frictionless security across various cloud providers and Kubernetes distributions.

Kubescape Security Frameworks

```
root@k8s-master-kubescape:~# kubescape list frameworks
```

Supported frameworks
AllControls
ArmoBest
DevOpsBest
MITRE
NSA
SOC2
cis-aks-t1.2.0
cis-eks-t1.2.0
cis-v1.23-t1.0.1

Figure 3: Available Security Frameworks

Kubescape CIS Scan

```
root@k8s-master-kubescape:~# kubescape scan framework cis-v1.23-t1.0.1 --submit
 ✅  Initialized scanner
 ✅  Loaded policies
 ✅  Loaded exceptions
 ✅  Loaded account configurations
 ✅  Accessed Kubernetes objects
Control: C-0169 100% |
 ✅  Done scanning. Cluster: kubernetes-admin-kubernetes
 ✅  Done aggregating results


Framework scanned: cis-v1.23-t1.0.1

┌─────────────────┬─────┐
│        Controls │ 121 │
│          Passed │ 37  │
│          Failed │ 37  │
│ Action Required │ 47  │
└─────────────────┴─────┘

Failed resources by severity:

┌──────────┬────┐
│ Critical │ 0  │
│     High │ 17 │
│   Medium │ 61 │
│      Low │ 1  │
└──────────┴────┘

Run with '--verbose'/'-v' to see control failures for each resource.

┌──────────┬──────────────────────────────────┬──────────────────┬─────────────────┬──────────────────┐
│ Severity │ Control name                     │ Failed resources │ All Resources   │ Compliance score │
├──────────┼──────────────────────────────────┼──────────────────┼─────────────────┼──────────────────┤
│ High     │ CIS-1.1.11 Ensure that the etcd data directory per... │ 0 │ 0 │ Action Required * │
│ High     │ CIS-1.1.12 Ensure that the etcd data directory own... │ 0 │ 0 │ Action Required * │
│ High     │ CIS-1.1.13 Ensure that the admin.conf file permiss... │ 0 │ 0 │ Action Required * │
│ High     │ CIS-1.1.14 Ensure that the admin.conf file ownersh... │ 0 │ 0 │ Action Required * │
│ High     │ CIS-1.1.19 Ensure that the Kubernetes PKI director... │ 0 │ 0 │ Action Required * │
│ High     │ CIS-1.1.20 Ensure that the Kubernetes PKI certific... │ 0 │ 0 │ Action Required * │
│ High     │ CIS-1.1.21 Ensure that the Kubernetes PKI key file... │ 0 │ 0 │ Action Required * │
│ High     │ CIS-1.2.1 Ensure that the API Server --anonymous-a... │ 1 │ 1 │ 0%                │
│ High     │ CIS-1.2.5 Ensure that the API Server --kubelet-cer... │ 1 │ 1 │ 0%                │
│ High     │ CIS-1.2.18 Ensure that the API Server --audit-log-... │ 1 │ 1 │ 0%                │
│ High     │ CIS-1.2.29 Ensure that the API Server --encryption... │ 1 │ 1 │ 0%                │
│ High     │ CIS-1.2.30 Ensure that encryption providers are ap... │ 0 │ 0 │ Action Required * │
│ High     │ CIS-4.1.7 Ensure that the certificate authorities ... │ 0 │ 0 │ Action Required * │
│ High     │ CIS-4.1.8 Ensure that the client certificate autho... │ 0 │ 0 │ Action Required * │
│ High     │ CIS-4.1.9 If the kubelet config.yaml configuration... │ 0 │ 0 │ Action Required * │
│ High     │ CIS-4.1.10 If the kubelet config.yaml configuratio... │ 0 │ 0 │ Action Required * │
│ High     │ CIS-4.2.1 Ensure that the --anonymous-auth argumen... │ 0 │ 0 │ Action Required * │
```

Figure 4: CIS Scan - Kubescape

Trivy Operator Auto Scan

```
You have installed Trivy Operator in the trivy-system namespace.
It is configured to discover Kubernetes workloads and resources in
all namespace(s).

Inspect created VulnerabilityReports by:

    kubectl get vulnerabilityreports --all-namespaces -o wide

Inspect created ConfigAuditReports by:

    kubectl get configauditreports --all-namespaces -o wide

Inspect the work log of trivy-operator by:

    kubectl logs -n trivy-system deployment/trivy-operator
```

Figure 5: trivy operator

```
halil@halil:~/trivy-operator$ kubectl get vulnerabilityreports -o wide
NAME                                REPOSITORY      TAG    SCANNER   AGE    CRITICAL   HIGH   MEDIUM   LOW   UNKNOWN
replicaset-nginx-599c4f6679-nginx   library/nginx   1.16   Trivy     2m1s   42         140    164      128   9
```

Figure 6: Nginx

Trivy Image Scan Example

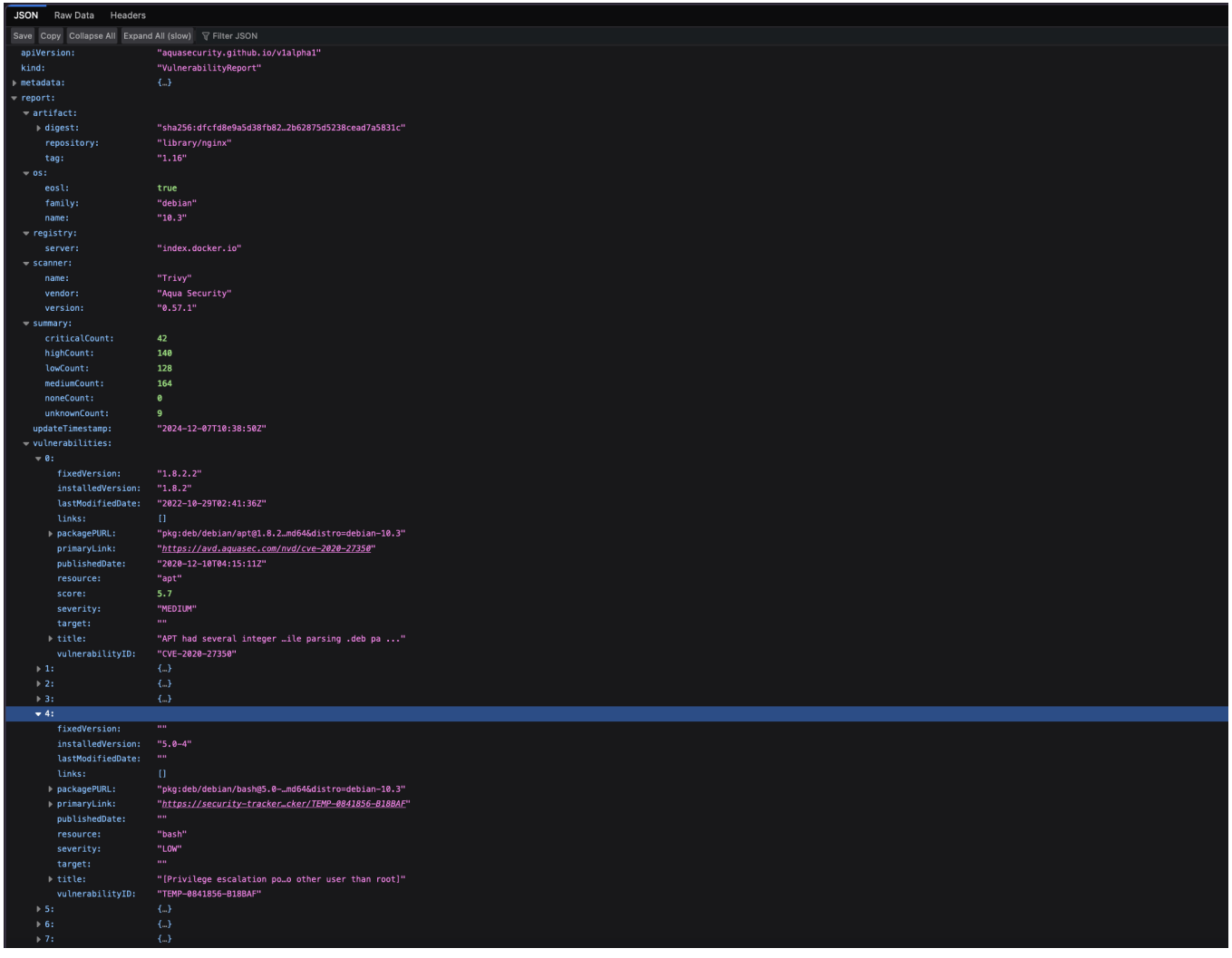


Figure 7: Trivy Report


A. Young, Y. Tamura, B. Laurie, Y. Zhang, R. Shapiro, and A. Chlipala, "The case for sandboxing linux applications with gvisor," in *Proceedings of the 11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud '19)*, USENIX Association, 2019.